

Audit de sécurité, de couverture fonctionnelle et d'état d'avancement

2 août 2026 · branche feat/v2-django-architecture · commit 99c09e3

Portée et méthode

Cet audit porte sur le dépôt à l'état du commit ci-dessus : backend Django (backend/), socle Dart partagé (packages/elcorazon_core) et les trois applications Flutter. Chaque constat est établi par lecture du code, exécution de la suite de tests ou de l'analyseur — aucun n'est déduit de la documentation, qui a elle-même été confrontée au code. Aucun test d'intrusion n'a été mené : ce qui suit est une revue de conception et de code, pas une validation d'exécution en conditions réelles.

Résumé exécutif

Le backend est solide et la migration qui vient de s'achever a fermé neuf failles exploitables depuis un poste de travail. Le tableau ci-dessous donne l'état des trois axes audités.

Axe	État	Constat principal
Sécurité — serveur	Faible	Refus par défaut, autorisation à trois étages, JWT RS256 avec rotation, webhooks signés en temps constant, liste des routes publiques vérifiée par un test.
Sécurité — applications	Critique	Les applications client et livreur embarquent encore les clés marchandes PayDunya et appellent le prestataire depuis l'appareil.
Couverture fonctionnelle	Moyenne	343 routes API sur 19 domaines. Le back-office est complet ; le client garde deux domaines non migrés.
Tests	Élevée	1 230 tests et 95,4 % de couverture côté serveur. Côté Flutter : 28 tests au total , et la CI n'exécute aucun test applicatif.
Avancement	Faible	7 phases sur 8 terminées. Reste le déploiement réel et la validation de bout en bout.

Le point qui commande tout le reste

Le principe qui structure l'architecture — **le client ne décide de rien qui engage l'enseigne** — est appliqué sans faille côté serveur et vient d'être imposé au back-office. Il ne l'est pas encore dans les deux applications distribuées au public : elles portent de quoi encaisser et rembourser. C'est le seul constat critique de cet audit, et il rend les autres secondaires tant qu'il n'est pas traité.

1. Sécurité

1.1 Constats ouverts

Sept constats, classés par gravité. La colonne « preuve » indique où le vérifier.

#	Gravité	Constat	Preuve
S-1	Critique	Clés marchandes PayDunya dans les applications client et livreur. PayDunyaService lit PAYDUNYA_MASTER_KEY, PRIVATE_KEY et TOKEN depuis le .env de l'app et les envoie en en-têtes HTTP directement à app.paydunya.com. Extraire ces clés d'un binaire distribué permet d'encaisser et de rembourser au nom de l'enseigne — sans permission, sans trace, sans plafond. Le backend a pourtant déjà la passerelle et les webhooks signés.	fastfood/lib/services/paydunya_service.dart:149 dely/lib/services/paydunya_service.dart:125
S-2	Élevée	Aucun test applicatif côté Flutter, et la CI n'en exécute pas. flutter-ci.yml lance flutter analyze sur les trois apps mais flutter test seulement sur le paquet partagé. Les apps comptent 0, 0 et 7 tests. Une régression sur un écran de paiement ou d'affectation ne serait rattrapée par rien.	.github/workflows/flutter-ci.yml:73-119
S-3	Moyenne	Clé Google Maps en clair dans les sources. Présente dans les web/index.html, les AndroidManifest.xml et AppDelegate.swift des trois apps. Inévitable pour une clé cliente, mais elle doit être <i>restreinte</i> (réfèrent HTTP, empreinte d'application, quotas) — sinon un tiers la consomme à votre facturation.	10 fichiers, dont admin/web/index.html
S-4	Moyenne	Lien résiduel vers l'ancien backend Node. fastfood route encore le géocodage, les itinéraires et un chemin de paiement web par un mandataire localhost:3000 ; admin ouvre un socket vers 10.0.2.2:3000 au démarrage, sans authentification. Ce service n'existe plus : le socket échoue en boucle, et le chemin de paiement web est mort.	fastfood/lib/config/api_config.dart:56 admin/lib/services/socket_service.dart:10
S-5	Faible	Gabarits d'environnement périmés. Les trois docs/env/*.env.example listent encore SUPABASE_URL et SUPABASE_ANON_KEY. Les valeurs sont vides — aucun secret n'a fuité — mais un nouvel arrivant configurerait une plateforme qui n'existe plus.	docs/env/*.env.example:4-5
S-6	Faible	Aucune configuration de production. docker-compose.yml est explicitement l'environnement de développement : mots de passe par défaut (elcorazon, minioadmin), ports base et Redis exposés sur l'hôte, TLS absent. prod.py est prêt et refuse de démarrer sans secrets, mais rien ne l'orchestre.	backend/docker-compose.yml:14-16, 98-99
S-7	Faible	Le jeton WebSocket peut voyager en chaîne de requête. Concession nécessaire — l'API navigateur interdit les en-têtes — mais un jeton en URL se retrouve dans les journaux d'accès et l'historique. L'en-tête est essayé en premier et le jeton d'accès expire en quinze minutes, ce qui borne l'exposition.	backend/common/consumers.py:87-103

1.2 Ce qui est bien tenu

La revue confirme un socle serveur sérieux. Ces points ne demandent aucune action ; ils sont listés parce qu'un audit qui ne relève que les manques donne une image fausse.

Domaine	Constat
Refus par défaut	DEFAULT_PERMISSION_CLASSES = [IsAuthenticated]. Chaque route publique le déclare explicitement, et un test d'architecture compare la liste réelle à une liste écrite : ouvrir une route par mégarde fait échouer la construction.
Autorisation	Trois étages (ADR-005) : type de compte porté par le jeton, permission nommée domaine.action issue d'un registre fermé , et cloisonnement par établissement appliqué dans les get_queryset — une ressource hors périmètre est introuvable, pas interdite.
Jetons	RS256, accès 15 min, rafraîchissement 30 j avec rotation et mise en liste noire. Les clés se lisent depuis un fichier monté ou l'environnement ; la production refuse de démarrer sans elles.
Paiements	Le webhook signé est la seule source de vérité de l'encaissement : le retour du client sur l'application n'écrit aucun état. Signature vérifiée en temps constant (hmac.compare_digest), authentification avant toute écriture.
Limitation de débit	Appliquée partout par défaut, avec des quotas dédiés : 5/min par identifiant tenté à l'authentification, 10/min sur la création de commande et l'initiation de paiement. NUM_PROXIES est réglé — sans lui, le limiteur par IP se contourne en forgeant X-Forwarded-For.
Temps réel	L'autorisation précède l'acceptation du socket (ADR-008). Il n'existe pas d'état « connecté mais pas encore autorisé » pendant lequel un message passerait.
Stockage	Compartiment privé, URL signées expirant en 15 minutes. Les pièces d'identité des livreurs ne sont jamais publiques.
Injection	Une seule requête SQL brute dans tout le backend : SELECT nextval('order_reference_seq'), statique et sans paramètre. Aucune interpolation de chaîne dans une requête.
Secrets côté client	Jetons rangés dans flutter_secure_storage (Keychain / Keystore). Aucune clé privée, aucun jeton JWT et aucun certificat Agora dans le dépôt.

2. Couverture fonctionnelle

Le backend expose **343 routes** sur 19 applications métier. La colonne « exploité par » indique quelles applications consomment réellement le domaine — un domaine servi mais inexploité est une dette, pas une fonctionnalité.

Domaine	Routes	Exploité par	Observation
catalog	38	client · back-office	Carte, options, avis, stock. Bibliothèque d'options réutilisables ajoutée.
loyalty	33	client · back-office	Points, récompenses, abonnements. Catalogue éditable depuis le back-office.
delivery	29	livreur · back-office	Courses, dossiers, notation, planning indicatif.
gamification	26	client · back-office	Succès, badges, défis.
geography	23	les trois	Pays, villes, zones et barèmes.
notifications	20	les trois	Historique, push FCM, campagnes de masse.
restaurants	18	client · back-office	Établissements, horaires, personnel.
group-carts	18	aucune	Backend complet, écran Flutter non réécrit.
orders	18	les trois	Cycle de vie complet, machine à états.
social	18	aucune	Backend complet, écrans non branchés.
administration	16	back-office	Clients, rôles, registre des permissions.
support	16	client	Réclamations, retours, tickets.
calls	15	client · livreur	Signalisation d'appel, jetons RTC signés côté serveur.
payments	12	client · back-office	Chemin client parallèle encore actif (S-1).
carts	10	client	Panier personnel, valorisation serveur.
analytics	8	back-office	Rapports, fiche client, chiffres de tête.
auth	7	les trois	Inscription, connexion, rotation, profil, appareils.
profiles	7	client	Adresses, préférences.
promotions	6	client · back-office	Codes, conditions, compteurs serveur.
tracking	2	livreur · client	Position émise par le livreur, diffusée par WebSocket.
search	1	back-office	Recherche transverse, permissions par famille.

2.1 État par application

Application	Fichiers	Modules partagés	Analyse	Tests	État
Client (fastfood)	181	34	0 err · 18 infos	0	Migrée sauf social et commande groupée
Livreur (dely)	74	6	0 err · 56 infos	0	Migrée

Application	Fichiers	Modules partagés	Analyse	Tests	État
Back-office (admin)	131	23	0 err · 2 infos	7	Migrée intégralement
Socle partagé	74	—	0 err	118	23 domaines couverts

Aucune des trois applications ne compte d'erreur ni d'avertissement. Les « infos » restantes sont des remarques de style sur des écrans que la migration n'a pas touchés (`use_build_context_synchronously`, `deprecated_member_use`).

Deux domaines servis mais inexploités

`group-carts` (18 routes) et `social` (18 routes) sont complets et testés côté serveur, mais aucune application ne les appelle : il reste à réécrire `group_order_screen.dart` et à brancher les groupes. Trente-six routes maintenues, documentées et testées que personne n'atteint — c'est le premier chantier fonctionnel à ouvrir.

3. État d'avancement

Phase	État	Contenu
1 — Analyse fonctionnelle	Terminée	Invariants métier nommés et numérotés (C1, L1–L4, P3, F1–F2).
2 — Architecture et ADR	Terminée	10 décisions d'architecture documentées et justifiées.
3 — Modèle de données	Terminée	Invariants défendus par des contraintes de base, pas seulement par du code.
4 — API REST v1	Terminée	343 routes, contrat OpenAPI, erreurs RFC 9457.
5 — Temps réel	Terminée	Channels, autorisation avant acceptation, rattrapage à la reconnexion.
7 — Tests et qualité	Terminée	Graphe de dépendances et contrat d'API vérifiés en CI, plancher de couverture à 92 %.
8 — Socle DevOps	Terminée	Docker, Compose, Nginx, deux workflows CI.
6 — Migration Flutter	Quasi terminée	Les trois apps sont sorties de Supabase. Restent \$3.6 (déploiement réel) et \$3.7 (validation de bout en bout).

3.1 Qualité mesurée

Indicateur	Valeur	Portail CI
Tests backend	1 230, tous verts	oui
Couverture backend	95,4 % (plancher : 92 %)	oui
ruff (style et bogues)	aucun écart	oui
ruff format	345 fichiers conformes	oui
mypy (strict)	245 fichiers, aucune erreur	oui
Tests du socle Dart	118, tous verts	oui
Analyse Flutter (3 apps)	0 erreur, 0 avertissement	oui
Tests applicatifs Flutter	7 au total	non

Le contraste est net : le backend est tenu par une suite dense et trois portails de qualité, les applications ne le sont que par l'analyseur statique. L'analyseur attrape ce qui ne compile pas ; il ne dit rien d'un écran qui affiche le mauvais prix.

3.2 Dette identifiée

- **Chemin de paiement client parallèle** — PayDunyaService subsiste dans les deux apps publiques (S-1). Le backend a déjà tout ce qu'il faut pour le remplacer : `/payments/{id}/initiate/` et les webhooks signés.
- **Deux domaines serveur sans écran** — `social` et `group-carts`, 36 routes maintenues et inatteignables.
- **Reliquat de l'ancien backend Node** — un mandataire de géocodage côté `fastfood`, un socket ouvert en boucle côté `admin` (S-4).
- **Aucun test d'écran** — y compris sur les parcours qui manient de l'argent (S-2).

- **Déploiement non éprouvé** — `prod.py` et la configuration Nginx existent, mais aucun déploiement réel n'a été exécuté ; FCM n'a pas été validé avec un vrai projet Firebase.
- **Documents d'inventaire datés** — signalés par des bandeaux, mais `ETAT_FONCTIONNALITES.md` et `SCHEMA_BDD_COMPLET.md` décrivent encore l'ancien monde dans leur corps.

3.3 Ordre de traitement recommandé

Rang	Action	Pourquoi maintenant
1	Retirer <code>PayDunyaService</code> des apps client et livreur ; brancher sur <code>/payments/</code> . Puis révoquer et régénérer les clés marchandes.	Elles sont dans des binaires distribués : les retirer du code ne suffit pas, les clés en circulation doivent être considérées comme compromises.
2	Restreindre la clé Google Maps (réfèrent, empreinte, quotas).	Quelques minutes de console, et cela borne une facturation aujourd'hui ouverte.
3	Ajouter <code>flutter test</code> à la CI, avec un premier jeu de tests sur les parcours de paiement et d'affectation.	Le portail existe déjà pour le paquet partagé ; l'étendre est mécanique.
4	Réécrire les écrans <code>social</code> et commande groupée.	Le backend est prêt, testé et documenté — c'est du travail d'écran seul.
5	Éprouver le déploiement : compose de production, TLS, MinIO, Celery beat, FCM réel.	Dernier verrou avant une mise en service.
6	Supprimer le mandataire Node et le socket résiduel.	Code mort qui échoue bruyamment et brouille les journaux.

Rapport établi par lecture du code et exécution des suites de tests et d'analyse. Les références de fichier et de ligne renvoient au commit 99c09e3.